

Memoria Técnica — Persistencia Políglota con Bases de Datos NoSQL

Documento de análisis, justificación arquitectónica, auditoría de código generado con IA, scripts de carga y consultas realizados para los sistemas críticos de EcoDrive: catálogo de vehículos, sesiones activas y red social de carpooling.

ALUMNOS Leandro Trista Hernandez Tomas Galdeano Williams Manuel Rey Laguarde Carlos Martín Codina	CURSO 2025-2026	PROYECTO EcoDrive · NoSQL
TECNOLOGÍAS MongoDB, Redis y Neo4j		

1. Introducción

EcoDrive es una plataforma de movilidad urbana sostenible que ofrece alquiler de patinetes eléctricos, bicicletas y coches compartidos. Debido a su crecimiento y a la diversidad de datos manejados por la plataforma, una única base de datos relacional tradicional deja de ser suficiente para cubrir todas las necesidades del sistema.

Por ello, se plantea una arquitectura de **persistencia políglota**, seleccionando para cada servicio el motor NoSQL más adecuado según el tipo de datos, el volumen de escrituras, la necesidad de escalado y el patrón principal de consulta.

En esta memoria se justifican las decisiones arquitectónicas, se documenta la auditoría crítica del código generado con IA y se incluyen los scripts finales de carga y consulta para cada motor NoSQL utilizado.

2. Justificación arquitectónica

La selección de cada sistema gestor se ha realizado en función del caso de uso concreto de cada servicio. La idea principal no es usar una sola base de datos para todo, sino elegir el modelo de persistencia que mejor resuelve cada necesidad.

Servicio	SGBD elegido	Tipo NoSQL	Justificación
Catálogo de vehículos y mantenimiento	MongoDB	Documental	Permite almacenar vehículos con atributos distintos y un historial de mantenimiento flexible dentro del mismo documento.
Sesiones activas y caché de batería	Redis	Clave-valor	Respuesta muy rápida en memoria, soporte nativo para TTL y estructuras eficientes para estado temporal y campos modificables.
Red social de carpooling	Neo4j	Grafos	Ideal para modelar relaciones entre usuarios, amistades, rutas compartidas y recomendaciones basadas en conexiones.

3. Elementos de diseño utilizados

MongoDB

Agregados y documentos

Se ha modelado un documento por vehículo, incrustando en él el historial de mantenimiento. Esto permite almacenar tipos de vehículos distintos sin un esquema rígido.

Campos incrustados: mantenimientos , piezas , coste , fecha .

Redis

Tipos de datos

Para sesiones temporales se ha usado expiración automática mediante TTL. Para la batería y estado de los vehículos se han utilizado estructuras HASH .

Claves: sesion:usuario:1 , vehiculo:veh001 .

Neo4j

Nodos y relaciones

Se han definido nodos Usuario , Ruta y relaciones con semántica clara como FRIEND_OF y JOINED .

Modelo relacional del grafo: usuarios conectados por amistad y por destino de viaje.

4. Prompts utilizados para la generación inicial con IA

Para obtener una primera propuesta de scripts se utilizaron prompts específicos para cada motor, indicando restricciones de modelado y evitando errores típicos de enfoque relacional.

Prompt para MongoDB

```
Genera un script MongoDB para una colección de vehículos de EcoDrive.  
Debe incluir patinetes, bicicletas y coches con atributos diferentes por tipo.  
Incluye historial de mantenimiento.  
Evita diseño relacional y usa documentos agregados si vehículo y mantenimiento se consultan juntos.
```

Prompt para Redis

```
Genera comandos Redis para sesiones temporales y caché de batería de vehículos de EcoDrive.  
Usa TTL para sesiones y evita usar solo strings para todo.  
Prioriza HASH si permite modificar campos individuales como batería.
```

Prompt para Neo4j

```
Genera un script Cypher para una red social de carpooling en EcoDrive.  
Crea usuarios, rutas y relaciones semánticas con dirección, como AMIGO_DE y VIAJA_A.  
Evita tratar los nodos como tablas relacionales.
```

5. Auditoría crítica del código generado por IA

Tras generar las primeras propuestas de código, se realizó una auditoría para detectar antipatrones NoSQL. El objetivo fue corregir diseños demasiado relacionales o estructuras poco adecuadas para cada motor.

Corrección técnica 1 — MongoDB: evitar normalización encubierta

La propuesta inicial separaba la información en dos colecciones: una para vehículos y otra para mantenimientos, relacionándolas mediante un identificador. Este diseño imitaba el modelo relacional y obligaba a reconstruir los datos mediante referencias.

Se corrigió incrustando el historial de mantenimiento dentro del documento del vehículo, ya que ambos datos se consultan habitualmente de forma conjunta. Esto mejora el rendimiento de lectura y respeta el concepto de agregado documental.

Corrección técnica 2 — Redis: uso de HASH en lugar de String

La IA proponía almacenar el estado de cada vehículo como un único string con SET. Aunque funcional, esta solución no es la más eficiente para Redis cuando se quiere modificar solo un campo concreto.

Para la representación de la flota de vehículos, se ha optado por la estructura HASH (HSET) en lugar de claves simples STRING (SET) por las siguientes razones:

- Agrupación Lógica y Cohesión:** Un vehículo en EcoDrive no es un dato único, sino un objeto con múltiples atributos (marca, batería, estado). El uso de HSET permite agrupar todos estos campos bajo una única clave (`vehículo:veh001`), manteniendo el espacio de nombres organizado y facilitando el acceso a la entidad completa.
- Flexibilidad y Operaciones Atómicas:** El uso de Hashes permite ejecutar comandos específicos como `HINCRBY` sobre un solo campo (la batería) sin tener que leer, modificar y volver a escribir todo el objeto. Esto garantiza la integridad de los datos en un entorno de alta concurrencia como es el seguimiento de una flota en tiempo real.

En EcoDrive, Redis no solo almacena datos, sino que gestiona el ciclo de vida del alquiler. Se utiliza la funcionalidad de expiración automática (TTL) para las sesiones de viaje. Mientras el vehículo envía señales de telemetría (latidos), el tiempo de expiración se refresca. Si la señal se pierde por más de 60 segundos, Redis elimina la clave, permitiendo que el sistema gestione automáticamente el fin del trayecto o posibles incidencias de conectividad.

Corrección técnica 3 — Neo4j: relaciones con semántica

Se revisó el modelado para asegurar que Neo4j no se usara como una tabla SQL disfrazada. Las relaciones se definieron con nombres y dirección clara, por ejemplo `(:Person)-[:FRIEND_OF]->(:Person)` y `(:Person)-[:JOINED]->(:Trip)`.

La IA generó varios CREATE para cargar los datos de la tabla, lo cual daba error al realizar consultas. Esto fue evitado atomizando el script de carga. Además, la IA no quiso añadir originalmente datos a las relaciones; se las añadí escribiendo las primeras a `(:Person)-[:JOINED]->(:Trip)` y luego el resto a partir de esas las autocompletó Copilot. Esto permitió crear un grafo con relaciones reales entre usuarios y rutas, y no solo nodos sin conexiones.

6. Scripts finales de carga y consultas

6.1. MongoDB — Catálogo de vehículos

Inserción de tres vehículos con atributos distintos y mantenimiento incrustado.

```
use ecodrive
```

```
db.vehiculos.insertMany([
  {
    _id: "veh001",
    tipo: "patinete",
    marca: "Xiaomi",
    modelo: "Mi Scooter Pro 2",
    autonomia_km: 45,
    limite_peso_kg: 100,
    disponible: true,
    mantenimientos: [
      {
        fecha: ISODate("2026-03-01T10:00:00Z"),
        tipo: "freno",
        piezas: ["pastillas de freno"],
        coste: 35
      }
    ]
  },
  {
    _id: "veh002",
    tipo: "bicicleta",
    marca: "Orbea",
    modelo: "Urban Ride",
    autonomía_km: 70,
    marchas: 7,
    disponible: true,
    mantenimientos: [
      {
        fecha: ISODate("2026-03-12T09:30:00Z"),
        tipo: "transmisión",
        piezas: ["cadena", "piñón"],
        coste: 55
      }
    ]
  },
  {
    _id: "veh003",
    tipo: "coche",
    marca: "Renault",
    modelo: "Zoe",
    autonomía_km: 300,
    tipo_combustible: "eléctrico",
    plazas: 5,
    disponible: false,
    mantenimientos: [
      {
        fecha: ISODate("2026-03-20T16:00:00Z"),
        tipo: "batería",
        piezas: ["módulo BMS"],
        coste: 320
      }
    ]
  }
])
```

Consulta de vehículos que tengan el atributo `tipo_combustible` .

```
db.vehiculos.find(
  { tipo_combustible: { $exists: true } },
  { _id: 1, tipo: 1, marca: 1, modelo: 1, tipo_combustible: 1 }
)
```

6.2. Redis — Sesiones y caché de batería

Creación de una sesión temporal con expiración automática en 60 segundos.

```
SET sesion:usuario:1 "token_seguro_alice_99" EX 60
TTL sesion:usuario:1
```

Registro de batería de tres vehículos distintos usando convención jerárquica con dos puntos.

```
HSET vehiculo:veh001 tipo "patinete" marca "Xiaomi" bateria 100 disponible true
HSET vehiculo:veh002 tipo "bicicleta" marca "Orbea" bateria 85 disponible false
HSET vehiculo:veh003 tipo "coche" marca "Renault" bateria 40 disponible false

HINCRBY vehiculo:veh001 bateria -5
HGETALL vehiculo:veh001
```

6.3. Neo4j — Red social de carpooling

Inserción de nodos y relaciones entre usuarios y rutas.

```
CREATE
(alice:Person {id:1, name:'Alice', age:28}),
(bob:Person {id:2, name:'Bob', age:31}),
(carla:Person {id:3, name:'Carla', age:26}),
(david:Person {id:4, name:'David', age:35}),
(elena:Person {id:5, name:'Elena', age:29}),
(frank:Person {id:6, name:'Frank', age:40}),
(grace:Person {id:7, name:'Grace', age:27}),
(hugo:Person {id:8, name:'Hugo', age:33}),
(irene:Person {id:9, name:'Irene', age:30}),
(javier:Person {id:10, name:'Javier', age:32}),

(madrid:Location {name:'Madrid'}),
(barcelona:Location {name:'Barcelona'}),
(valencia:Location {name:'Valencia'}),
(sevilla:Location {name:'Sevilla'}),
(zaragoza:Location {name:'Zaragoza'}),
(bilbao:Location {name:'Bilbao'}),

(alice)-[:FRIEND_OF]->(bob),
(bob)-[:FRIEND_OF]->(alice),

(alice)-[:FRIEND_OF]->(carla),
(carla)-[:FRIEND_OF]->(alice),

(bob)-[:FRIEND_OF]->(david),
(david)-[:FRIEND_OF]->(bob),

(carla)-[:FRIEND_OF]->(elena),
(elena)-[:FRIEND_OF]->(carla),

(carla)-[:FRIEND_OF]->(bob),
(bob)-[:FRIEND_OF]->(carla),

(elena)-[:FRIEND_OF]->(grace),
(grace)-[:FRIEND_OF]->(elena),

(frank)-[:FRIEND_OF]->(hugo),
(hugo)-[:FRIEND_OF]->(frank),

(hugo)-[:FRIEND_OF]->(irene),
(irene)-[:FRIEND_OF]->(hugo),

(irene)-[:FRIEND_OF]->(javier),
(javier)-[:FRIEND_OF]->(irene),

(t1:Trip {id:101, date:'2026-05-10'}),
(t2:Trip {id:102, date:'2026-05-11'}),
(t3:Trip {id:103, date:'2026-05-12'}),
(t4:Trip {id:104, date:'2026-05-13'}),
(t5:Trip {id:105, date:'2026-05-14'}),
(t6:Trip {id:106, date:'2026-05-15'}),
(t7:Trip {id:107, date:'2026-05-16'}),

(t1)-[:STARTS_AT]->(madrid),
(t1)-[:ENDS_AT]->(barcelona),

(t2)-[:STARTS_AT]->(madrid),
(t2)-[:ENDS_AT]->(valencia),

(t3)-[:STARTS_AT]->(sevilla),
(t3)-[:ENDS_AT]->(madrid),

(t4)-[:STARTS_AT]->(zaragoza),
(t4)-[:ENDS_AT]->(barcelona),

(t5)-[:STARTS_AT]->(madrid),
(t5)-[:ENDS_AT]->(barcelona),
```

```
(t6)-[:STARTS_AT]->(bilbao),
(t6)-[:ENDS_AT]->(madrid),

(t7)-[:STARTS_AT]->(madrid),
(t7)-[:ENDS_AT]->(sevilla),

(alice)-[:JOINED {rating: 5}]->(t1),
(bob)-[:JOINED {rating: 4}]->(t1),

(carla)-[:JOINED {rating: 2}]->(t2),
(david)-[:JOINED {rating: 3}]->(t2),

(elena)-[:JOINED {rating: 4}]->(t3),
(frank)-[:JOINED {rating: 5}]->(t3),

(grace)-[:JOINED {rating: 2}]->(t4),
(hugo)-[:JOINED {rating: 4}]->(t4),

(irene)-[:JOINED {rating: 3}]->(t5),
(alice)-[:JOINED {rating: 5}]->(t5),

(javier)-[:JOINED {rating: 4}]->(t6),
(bob)-[:JOINED {rating: 4}]->(t6),

(carla)-[:JOINED {rating: 2}]->(t7),
(grace)-[:JOINED {rating: 3}]->(t7);
```

Consulta de personas que realizan el mismo viaje sin esa persona

```
MATCH (user:Person {name:'Bob'})
MATCH (candidate:Person)-[:JOINED]->(trip:Trip)
MATCH (user)-[:JOINED]->(utrip:Trip)
MATCH (trip)-[:STARTS_AT]->(start)
MATCH (trip)-[:ENDS_AT]->(end)
MATCH (utrip)-[:STARTS_AT]->(ustart)
MATCH (utrip)-[:ENDS_AT]->(uend)

WHERE candidate <> user AND uend = end AND ustart = start AND trip <> utrip

RETURN DISTINCT candidate.name;
```

7. Ejecución y visualización

Para validar la implementación, se deben incluir capturas de pantalla de la ejecución de los scripts y de los resultados de las consultas en cada motor. Estas capturas pueden obtenerse desde consola o desde interfaces visuales como Mongo Express, RedisInsight o Neo4j Browser.

7.1. Capturas de MongoDB

7.1.1. Carga de datos

Ejecuto el código para cargar el script con la información de los datos que voy a insertar a la base de datos:

```
docker exec -i mongodb-server mongoimport --db ecodrive --collection vehiculos --file /src/ecodrive-seed.json --jsonArray
```



```
leandro@MacbookLeo:~/BasesDatos/Teoria/UT8_NoSQL/Recursos/Dockers/mongodb$ sudo docker exec -i mongodb-server mongoimport --db ecodrive --collection vehiculos --file /src/ecodrive-seed.json --jsonArray;
2026-05-21T23:56:21.180+0000 connected to: mongodb://localhost/
2026-05-21T23:56:21.213+0000 3 document(s) imported successfully. 0 document(s) failed to import.
```

7.1.2. Consulta de vehículos

Consulta de vehículos con el atributo tipo_combustible:

```
ecodrive> db.vehiculos.find(
|   { tipo_combustible: { $exists: true } },
|   { _id: 1, tipo: 1, marca: 1, modelo: 1, tipo_combustible: 1 }
| )
[
|   {
|     _id: 'veh003',
|     tipo: 'coche',
|     marca: 'Renault',
|     modelo: 'Zoe',
|     tipo_combustible: 'eléctrico'
|   }
| ]
ecodrive>
```

7.2. Capturas de Redis

7.2.1. Inicialización y acceso

Levantar el contenedor Docker y entrar a la consola de Redis:

```
root@ubuntu:~$ docker run -d --name redis_server -p 6379:6379 redis:alpine
Unable to find image 'redis:alpine' locally
alpine: Pulling from library/redis
6a0ac1617861: Pull complete
9fba6c0845a7: Pull complete
f668659e960c: Pull complete
2156d2090f3d: Pull complete
df16ec49200a: Pull complete
4f4fb700ef54: Pull complete
111abd5c7b8c: Pull complete
Digest: sha256:d146f83b1e0f02fc27c26a50cee39338c736674c5959db84363e6ae3cd9e02d2
Status: Downloaded newer image for redis:alpine
f3a0375141838d973f7e0573a719b6c7dc02676f666b63eac20ee2d146865f6b
root@ubuntu:~$
```

```
root@ubuntu:~$ docker exec -it redis_server redis-cli
127.0.0.1:6379> 
```

7.2.2. Gestión de vehículos

Carga de vehículos, consulta de datos y modificación de batería:

```
root@ubuntu:~$ docker exec -it redis_server redis-cli
127.0.0.1:6379> HSET vehiculo:001 tipo "patinete" marca "Xiaomi" bateria 100 estado "disponible"
(integer) 4
127.0.0.1:6379> HSET vehiculo:002 tipo "bici" marca "Orbea" bateria 85 estado "en_uso"
(integer) 4
127.0.0.1:6379> HSET vehiculo:003 tipo "coche" marca "Tesla" bateria 40 estado "en_carga"
(integer) 4
root@ubuntu:~$ docker exec -it redis_server redis-cli
127.0.0.1:6379> HGETALL vehiculo:002
1) "tipo"
2) "bici"
3) "marca"
4) "Orbea"
5) "bateria"
6) "85"
7) "estado"
8) "en_uso"
127.0.0.1:6379>
127.0.0.1:6379> HINCRBY vehiculo:003 bateria -5
(integer) 35
127.0.0.1:6379> HGET vehiculo:003 bateria
"35"
127.0.0.1:6379> 
```

7.2.3. Gestión de sesiones

Creación de token de usuario y asignación de expiración (TTL):

```
127.0.0.1:6379> SET sesion:usuario:1 "token_seguro_alice_99" EX 60
OK
127.0.0.1:6379> TTL sesion:usuario:1
(integer) 26
127.0.0.1:6379> TTL sesion:usuario:1
(integer) 19
127.0.0.1:6379> TTL sesion:usuario:1
(integer) 17
127.0.0.1:6379> 
```

7.3. Capturas de Neo4j

7.3.1. Visualización del grafo

Carga de los nodos (personas y rutas) y sus relaciones:

```
95 (t7)-[:STARTS_AT]->(madrid),
96 (t7)-[:ENDS_AT]->(sevilla),
97
98 (alice)-[:JOINED {rating: 5}]->(t1),
99 (bob)-[:JOINED {rating: 4}]->(t1),
100
101 (carla)-[:JOINED {rating: 2}]->(t2),
102 (david)-[:JOINED {rating: 3}]->(t2),
103
104 (alena)-[:JOINED {rating: 4}]->(t3),
105 (frank)-[:JOINED {rating: 5}]->(t3),
106
107 (grace)-[:JOINED {rating: 2}]->(t4),
108 (hugo)-[:JOINED {rating: 4}]->(t4),
109
110 (irene)-[:JOINED {rating: 3}]->(t5),
111 (alice)-[:JOINED {rating: 5}]->(t5),
112
113 (javier)-[:JOINED {rating: 4}]->(t6),
114 (bob)-[:JOINED {rating: 4}]->(t6),
115
```

Created 23 nodes, created 46 relationships, set 64 properties, added 23 labels
Completed after 483 ms

7.3.2. Consulta en el grafo

Resultado de la consulta Cypher para encontrar personas realizando el mismo viaje:

```
1 MATCH (user:Person {name:'Bob'})
2 MATCH (candidate:Person)-[:JOINED]->(trip:Trip)
3 MATCH (user)-[:JOINED]->(utrip:Trip)
4 MATCH (trip)-[:STARTS_AT]->(start)
5 MATCH (trip)-[:ENDS_AT]->(end)
6 MATCH (utrip)-[:STARTS_AT]->(ustart)
7 MATCH (utrip)-[:ENDS_AT]->(uend)
8
9 WHERE candidate <> user AND uend = end AND ustart = start AND trip <> utrip
10
11 |
12
13 RETURN DISTINCT candidate.name;
```

Table RAW

candidate.name
"Alice"
"Irene"

8. Conclusión

La práctica demuestra que una arquitectura NoSQL eficaz no consiste en sustituir una base de datos relacional por otra única tecnología, sino en seleccionar el motor más adecuado para cada necesidad concreta. En este caso, MongoDB ofrece flexibilidad documental para el catálogo, Redis resuelve el estado temporal con gran velocidad y Neo4j modela de forma natural la red social de usuarios.

Además, la auditoría del código generado con IA ha permitido corregir varios antipatrones habituales, mejorando el diseño final y alineándolo con los principios de modelado propios de cada sistema NoSQL.